音廻 ／ コード解説

敵への攻撃と、ロングの盾のしくみ

叩いた時の破片とレーザー、着弾、ダメージと撃破、ロングの盾が張られてヒビが入り割れるまでを、ファイルごとに「何をしているか」と「なぜそれでできるか」でまとめた資料。

2026年11月10日時点の move_lane ブランチのコードを読んで作成

---

1 全体の地図（どのファイルが何を担当しているか）

2 敵：HP・ダメージ・撃破・光らせる（Enemy）

3 叩いた時の攻撃：破片とレーザー（HitBurstFx）

4 着弾の演出（ImpactFx）と、今は止めている攻撃の光（AttackOrbs）

5 ロングの盾：張る・ヒビ・割れる（LongShield ＋ ChartObject）

6 必殺技：ゲージ・弱点・ビーム（SpecialMove / WeakPoint / SpecialBeam）

7 全部を支える道具（LongBeamRibbon / RibbonShapes / GameEvent）

8 読んでいて気になった所

1. 全体の地図

攻撃と盾の処理は、いくつものクラスに分かれている。分け方の決まりは一つで、**「見た目だけのクラス」と「ダメージや音を決める ChartObject」を分ける**こと。見た目のクラスは「何かが起きた」をコールバックやイベントで知らせるだけで、ダメージは入れない。

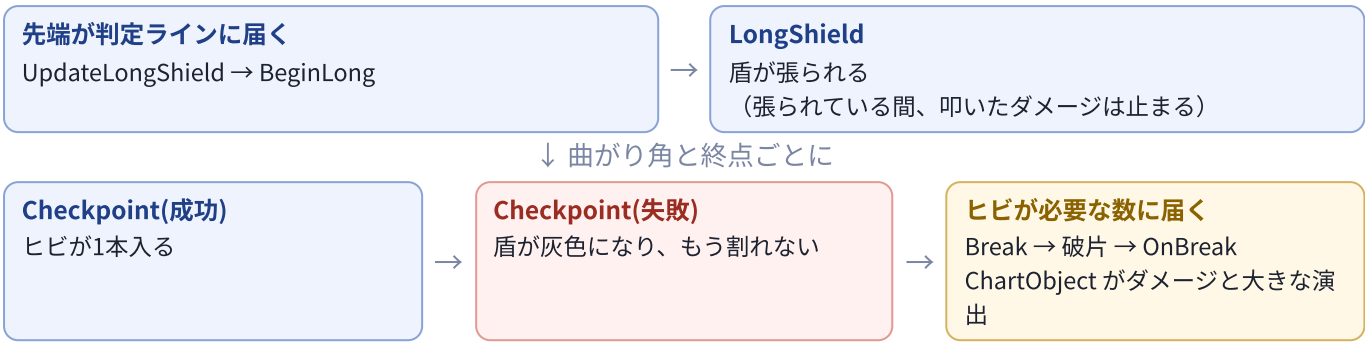
ファイルと役割

ファイル	担当	ダメージを入れるか
`Enemy`	敵のHP、瀕死・撃破・爆発の状態、攻撃を受けた時の揺れと光	受ける側（`Damage`）
`ChartObject`	判定を受けて、ダメージ・音・演出のきっかけを全部決める司令塔	**入れる**
`HitBurstFx`	叩いたノーツの破片と、拍で飛ぶレーザー	入れない（届いたら知らせる）
`ImpactFx`	着弾の輪・六角形・火花	入れない
`AttackOrbs`	判定円から敵へ飛ぶ光（今は止めている）	入れない
`LongShield`	ロングの盾。張る・ヒビ・割れる・波紋・破片	入れない（割れたら知らせる）
`SpecialMove`	必殺技の状態とゲージ・チャージ量の計算	入れない（計算式だけ持つ）
`WeakPoint`	弱点のキーと場所、狙いとロック	入れない
`SpecialBeam`	必殺技のビーム（帯・24本の光・雷）	入れない（当たったら知らせる）
`GameEvent`	「叩いた」「拍の頭」「当たった」などを、聞きたいクラスに一斉に知らせる放送局	—
`LongBeamRibbon` `RibbonShapes`	点の並びを光の帯にして描く道具。攻撃・盾・火花の見た目は全部これで描いている	—

叩いてから敵が光るまで



ロングを押してから盾が割れるまで



2. 敵：HP・ダメージ・撃破・光らせる（Enemy）

HP と状態

状態	いつ	見た目
Alive	最初	ゆっくり回る
Dying（瀕死）	HP が 1（`kDyingHp`）まで減った	0.6 秒ごとに赤く点滅。回転がゆっくりになる
Defeated（とどめ）	最後のノーツの判定が終わった時、瀕死なら `Finish()`	真っ白に光り、だんだん強く揺れる
（爆発）	とどめから 0.9 秒（`kExplodeTime`）	消えて、`OnExploded` で大きな着弾・揺れ・閃光

```
void Damage(float amount)
{
        if (m_State == State::Defeated) return;      //倒された後の攻撃は無効
        m_Hp -= amount;
        if (m_Hp < kDyingHp) m_Hp = kDyingHp;         //とどめの前は瀕死で止める
        m_HitTimer = 0.25f;                           //0.25秒揺れる
        if (m_State == State::Alive && m_Hp <= kDyingHp)
        {
                m_State = State::Dying;
                if (m_OnDying) m_OnDying();
        }
}
```

なぜこうしているか：HP は 1 より下がらないので、どれだけ攻撃しても曲の途中で倒れない。倒れるのは「最後のノーツを叩き終えた時」だけになる。ダメージの量をどう調整しても、**撃破は必ず曲の終わりに起きる**。音ゲーとして、曲の途中で戦いが終わってしまうのを防げる。

入るダメージの一覧（HP は 100）

いつ	どこ	量	条件
叩くたび	`OnJudge`	`60 ÷ m_TotalNote`（全部叩くと合計 60）	盾が張られている間は入らない
ロングを押し切った	`SetOnLongEnd`	`40 ÷ ロングの本数`	最後まで押していた時
盾が割れた	`SetOnBreak`	`40 ÷ ロングの本数 × その盾のロングの本数`	ヒビが全部入った時
必殺技	`SetOnHit`	`15 × (0.3 + 0.7 × チャージ量) × 倍率`	弱点なら ×2.0、防がれたら ×0.3

m_TotalNote は「タップの数 ＋ ロングのチェックポイントの数」。ダメージは叩いた瞬間に入り、レーザーが当たった時ではない（見た目と計算を分けているため）。

敵を一瞬光らせる（Flash）

```
void Enemy::Flash(const Vector3& color, float strength, float duration)
{
        if (strength < CurrentFlash()) return;   //今光っている方が強ければ、そちらを残す
        m_FlashColor = color; m_FlashStrength = strength;
        m_FlashDuration = std::max(0.01f, duration); m_FlashTime = m_FlashDuration;
}
float Enemy::CurrentFlash() const
{
        float k = m_FlashTime / m_FlashDuration;    // 1 → 0
        return m_FlashStrength * k * k;             // 2乗で、最初に強く、すぐ元に戻る
}
```

Draw で、今の光り具合をシェーダー（`enemyFlash.hlsl`）に送る。シェーダーは「元の色」と「光らせたい色」を、光り具合の割合で混ぜる（`lerp`）。

> **なぜこうしているか：**攻撃が連続で当たると、Flash が何度も呼ばれる。弱い光（普通の着弾）が強い光（弱点・とどめ）を上書きすると、大事な瞬間が見えなくなる。「今より弱い光は無視する」の1行で、強い演出が必ず残る。2乗で減らすのは、「パッ」と光ってすぐ戻る感じにするため（まっすぐ減らすと、ぼんやり長く残って見える）。

3. 叩いた時の攻撃：破片とレーザー（HitBurstFx）

叩いたノーツがその場で判定の色の破片になってはじけ、次の拍の頭で破片の一つ一つが敵のまわりへ飛び、そこからまっすぐなレーザーになって敵に刺さる。

イベントの聞き方（Init）

```
m_JudgeSub = GameEvent::OnJudge().Subscribe([this](const GameEvent::Judge& e)
        {
                if (e.isLong) return;                                   //ロングの途中は出さない
                SpawnBurst(e.pos + Vector3(0.0f, kLift, 0.0f), e.rank);  //判定円の少し上ではじける
        });
m_BeatSub = GameEvent::OnBeat().Subscribe([this](const GameEvent::Beat& e)
        {
                if (e.index % kFireEveryBeats == 0) FireReady();        //拍の頭で撃つ
        });
```

なぜこうしているか：HitBurstFx は ChartObject のことを何も知らない。「叩いた」と「拍の頭」の放送を聞くだけで動く。そのため、この演出を足すのに判定の処理（ChartObject）を変えずに済んだ。消したい時も、Stage1Scene で追加している行を消すだけでよい。

流れと状態

段階	何をしているか	関数
はじける	判定ごとに破片の数と勢いを変えて、上の方へばらばらに飛ばす（WEAK 14個／HIT 10個／GRAZE 6個）	`SpawnBurst`
漂う	速さを毎フレーム減らし（`kDrag`）、少し浮かせて、sin と cos で小さく揺らす	`Update`
撃つ	拍の頭で、はじけてから `kMinHover` 秒以上たったかたまりを撃つ。拍が来なくても `kMaxWait` 秒で撃つ（保険）	`FireReady` `FireBurst`
発射位置へ飛ぶ	破片が、敵のまわりの発射位置へ `kDashTime` 秒ですっと飛ぶ	`Draw`
レーザー	発射位置から敵へ、まっすぐな線。先に光の玉	`Draw` `DrawFlare`
届いた	かたまりの最初の1本だけが `OnAttackHit` を放送する	`Update`

発射位置の決め方：レーンをまたがない理由

```
float side = (from.x >= m_Target.x) ? 1.0f : -1.0f;          //右のレーンの破片は右側、左は左側
float rad  = XMConvertToRadians(Rand(-15.0f, 85.0f));        //真横の少し下 ～ ほぼ真上
Vector3 dir(side * cos(rad), sin(rad), Rand(-0.3f, 0.2f));
l.launch = l.to + dir * Rand(kLaunchDistMin, kLaunchDistMax);  //敵から600～900離れた所
```

> **なぜレーンをまたがないか：**破片からそのまま敵へ撃つと、線がレーンの上を通ってしまう。そこで一度、敵のまわりの発射位置へ飛ばしてから撃つ。発射位置は「破片と同じ側」で、「真下（レーンのある方）を除いた角度」にしか置かない。なので、レーザーは敵のまわりのいろんな方向から刺さるが、レーンの上は通らない。曲線（ベジェ）を使わず、2点の間の割合（`Lerp`）だけで動かしている。

一斉射撃に見せる工夫

- **撃つ時刻を少しずつずらす：**破片ごとに 0〜`kFireSpread` 秒の待ちを付ける（`l.age = -delay`。マイナスの間は描かない）。
- **「当たった」はかたまりに1回：**14本全部で着弾の演出を出すと、火花が多すぎて重く、画面もうるさい。最初の1本（`notify`）だけが知らせる。
- **知らせるのはループの外：**届いたものを `arrived` に集めてから、ループが終わった後で放送する。放送を聞いた先で何かが足されても、回している最中のリストが壊れない。

光の玉（DrawFlare）の作り方

「真ん中が一番濃く、両端で消える」帯（点3つ）を、向きと太さを変えて5本重ねている。

帯	長さ	太さ	役目
2本（横・縦）	size × 0.9	size × 1.8	太くて短い → ぼんやりした丸い光
2本（横・縦）	size × 2.8／2.0	size × 0.12	細くて長い → 十字の光の筋
1本	size × 0.2	size × 0.4	小さな白い芯

向きは `RibbonShapes::CameraBasis` で求めた「画面の右と上」を使うので、どこから見ても十字に見える。飛んでいる間は sin で少し瞬かせ、届いた時は大きく広がって消える（`kHitFlashTime`）。

今の調整値（パソコンのコードの値）

値	今	意味
`kMinHover`	0.1	はじけてから、最低これだけ待ってから撃つ
`kFireEveryBeats`	1	何拍ごとに撃つか
`kFireSpread`	0.4	破片ごとの撃つ時刻のずれの最大
`kDashTime` / `kLaserTime`	0.12 / 0.18	発射位置へ飛ぶ秒数 ／ 敵に届く秒数
`kLaunchDistMin/Max`	600 / 900	発射位置の、敵からの距離
`kMaxBursts` / `kMaxLasers`	4 / 10	同時に出せる数（超えたら古いものから消す）
`RankColor`	金／水色／緑	WEAK／HIT／GRAZE の色（GRAZE のコメントは「紫」のまま）

4. 着弾の演出（ImpactFx）と、今は止めている攻撃の光（AttackOrbs）

「当たった」を聞く所（ChartObject::Init）

```
m_AttackHitSub = GameEvent::OnAttackHit().Subscribe([this](const GameEvent::AttackHit& e)
        {
                if (m_LongShield.IsUp()) { m_LongShield.Ripple(e.pos); return; }   //盾にはじかれた
                bool critical = (e.rank == 0);
                if (critical) m_ImpactFx.SpawnWeakHit(e.pos, e.color);
                else          m_ImpactFx.SpawnSmallHit(e.pos, e.color);
                if (m_EnemyRef) m_EnemyRef->Flash(...);                              //敵を一瞬光らせる
        });
```

なぜ ChartObject が聞くのか：着弾の演出（ImpactFx）、盾（LongShield）、敵（Enemy）を持っているのが ChartObject だから。HitBurstFx はそれらを知らないまま「当たった」を放送し、知っている ChartObject が受け取って演出を出す。放送する側と聞く側が逆になっているだけで、同じ仕組み。

ImpactFx：輪と火花

着弾は「広がる輪（円か六角形）」と「飛び散る火花」の組み合わせだけでできている。種類ごとに、数と大きさを変えているだけ。

関数	使う所	中身
`SpawnSmallHit`	普通の着弾	小さい輪 ＋ 火花 5（明るさ半分）
`SpawnWeakHit`	WEAK の着弾	金の輪 ＋ 六角形 ＋ 上向きの火花 12
`SpawnCritical`	必殺技が弱点に／爆発	白い閃光 ＋ 大きな衝撃の輪 ＋ 六角形 ＋ 火花たくさん
`SpawnBlocked`	必殺技が防がれた	青白い六角形の盾 ＋ 来た方へはね返る火花

```
//輪：最初に速く広がって、だんだんゆっくり（1-(1-k)^3)。広がりながら薄くなる
float e = 1.0f - (1.0f - k) * (1.0f - k) * (1.0f - k);
float radius = r.r0 + (r.r1 - r.r0) * e;
//火花：重力で落ち、空気の抵抗で遅くなる
s.vel.y -= kGravity * dt;
s.vel  *= max(0, 1 - kDrag * dt);
s.pos  += s.vel * dt;
//火花の描き方：尾 = 今の位置 - 速さ × 0.035秒。速い火花ほど長い線になる
```

なぜ上限があるか：叩くたびに増えるので、上限（`kMaxRings=32`、`kMaxSparks=240`）を超えたら古いものから消す。上限がないと、密な所で描く数が増え続けて重くなる。

AttackOrbs：判定円から敵へ飛ぶ光 今は止めている

以前の攻撃の光。今は OnJudge の条件が `(!isLong && kTapAttackOrbs)` で、`kTapAttackOrbs = false` なので、一度も飛ばない（タップは HitBurstFx のレーザーに置き換え、ロングの途中は帯が邪魔なので止めた）。コードは残っているので、仕組みだけ書いておく。

- **3次ベジェ曲線：**判定円 → 判定円の真上で外側 → 敵の少し上で外側 → 敵、の4点。左手のレーン（ＳＤＦ）は左へ、右手（ＪＫＬ）は右へふくらむ。外側のレーンほど大きくふくらむ。
- **尾を引く：**先の位置 `head = k × (0.55 + 0.45k)`（だんだん速くなる）、尾は `head - kTrail`。その間だけ点を並べて帯にする。
- **届いたら知らせる：**届いたものを集めてから、ループの外で `OnArrive` を呼ぶ（HitBurstFx と同じ理由）。

これを止めたことで、ダメージは変わらない（ダメージは OnJudge で叩いた瞬間に入っているため）。なくなったのは「ロングの間、光が盾にはじかれて波紋が出る」演出だけ。

5. ロングの盾：張る・ヒビ・割れる（LongShield ＋ ChartObject）

「ロングを押している間が暇」への答え。ロングの間、敵が盾を張る。曲がり角でうまく押し変えるたび、押し切った時にヒビが入り、全部入ると割れてダメージ。1つでも失敗すると、その盾は割れないだけで、罰はない。

判定する側：ChartObject::UpdateLongShield

いつ	何をするか
先端が判定ラインに届いた	`BeginLong(曲がり角の数 + 1)`。ヒビが入るチャンスは「曲がり角ごと ＋ 終点」
曲がり角	曲がった時刻を過ぎたら、新しいレーンを押しているか調べる。押していなくても、`kTurnGrace`（0.12秒）までは待つ。押せていれば `Checkpoint(true)`、猶予が過ぎたら `Checkpoint(false)`
終点	「1つ前のフレームで押せていたか」（`shieldLastHeld`）で `Checkpoint`。そして `EndLong`

> **なぜ「1つ前のフレーム」で見るのか：**終点のフレームでは、ロングの判定がもう終わっていて、押していても「押している」と数えられないことがある。毎フレーム「今押せているか」を覚えておき、終わったフレームではその値を使うことで、最後まで押していた人が正しく成功になる。

盾の状態（LongShield）

状態	いつ	見た目
Down	盾なし	何も出ない
Rising	`BeginLong` から 0.18 秒	少し大きく出てから戻る（ポンと張られる）
Up	張られている	六角形の縁とハチの巣。ヒビが入るたびに少し震える
Breaking	ヒビが全部入った	0.15 秒で外へ弾けて消え、破片が飛ぶ
Fading	割れないままロングが全部終わった	0.3 秒で静かに消える

```
void LongShield::BeginLong(int checkpoints)
{
        if (!IsUp()) { /* 新しい盾を張る（数を0に戻す） */ }
        m_Active++;                                 //今押されているロングの本数
        m_Longs++;                                  //この盾に関わったロングの本数
        m_Required += std::max(1, checkpoints);     //割るのに必要なヒビの数を足していく
}
```

> **なぜ足していくのか：**盾が張られている間に別のロングが始まったら、新しい盾を張らずに、同じ盾に「必要なヒビの数」を足す。ロングが重なっても盾は1枚のまま。割れた時のダメージは「関わったロングの本数」分になる（`OnBreak(m_Longs, ...)`）。全部のロングが終わった時（`m_Active == 0`）に、割れていなければ消える。

失敗した時

`Checkpoint(false)` で `m_Failed = true`。盾の色が一瞬赤くなってから灰色になり、それ以降はヒビが入らない。ロングが終わると静かに消える。プレイヤーへの罰（ダメージを受けるなど）はない。

盾の上の座標：2D で考えて、最後に 3D へ

```
Vector3 LongShield::ToWorld(float x, float y, float z) const
{
	Vector3 right, up, toCam;
	Basis(right, up, toCam);                    //敵から見た、カメラの方を向いた面の「右」「上」
	Vector3 c = m_Center + up * kLift;          //盾の中心（敵の中心から少し上）
	return c + right * (x * kStretchx) + up * y + toCam * z;
}
```

> **なぜこうしているか：**ヒビや六角形を 3D のまま計算すると難しい。そこで「盾の面の上の 2D 座標（x, y）」で全部を考え、描く直前に `ToWorld` で 3D に直している。面の向きは `CameraBasis`（外積で「カメラへの向き」と「真上」の両方に直角な向き＝画面の右を出す）で決めるので、カメラが動いても盾はいつも正面を向く。横だけ 1.3 倍（`kStretchx`）にして、少し横長の盾にしている。

六角形の縁とハチの巣

- 外側の縁：太い線（幅10）と、ぼんやりした光（幅28、薄く）の2回描き。
- 中のハチの巣：真ん中に1つ、まわりに6つ。小さい六角形の大きさを a とすると、隣の中心までの距離は $a \times \sqrt{3}$。この距離で並べると、六角形がすき間なく並ぶ。

盾の下の方を薄くする（VerticalFade）

盾の y が上の方（`kFadeTop` より上）なら濃さ 1、一番下（`kFadeBottom`）で `kFadeMin`（0.15）になるように、smoothstep でなめらかにつなぐ。縁・ヒビ・破片の全部にかける。

> **なぜこうしているか：**盾の下の方は、レーンやノーツと重なりやすい。企業のフィードバック「ノーツの周りは見やすく」に合わせて、重なる所ほど薄くしている。

5（続き）. ヒビと破片

ヒビの起点：黄金角でずらす

```
m_LastCrackAngle += 2.39996f + Rand(-0.3f, 0.3f);   //黄金角（約137.5度）＋ 少しのずれ
float r0 = kRadius * Rand(0.1f, 0.55f);
c.origin = (sin(角度) × r0, cos(角度) × r0);
```

> **なぜ黄金角か：**ランダムに起点を選ぶと、同じ場所にヒビが固まることがある。前の起点から 137.5度 ずつ回すと、何本入れても起点が盾の全体に均等に散らばる（ひまわりの種の並びと同じ考え方）。少しだけランダムを足して、機械的に見えないようにしている。

ヒビの線：ガラスらしいぎざぎざと枝分かれ

- 起点から3〜4本の「腕」を放射状に伸ばす。必要なヒビが多い（6より多い）盾では2〜3本にして、線だらけにならないようにする。
- `AddCrackLine`：少しずつ進みながら、向きを小さく曲げる（3割の確率で大きめに折れる）。ガラスのヒビは「ほぼまっすぐで、ところどころ折れる」ので。
- 盾の縁（半径の 95%）に着いたら止める。
- 途中で1回だけ枝分かれすることがある（自分自身をもう一度呼ぶ＝再帰。`depth` で深さを数え、枝からさらには分かれない）。
- 隣の腕どうしを、少しずらした線でつなぐ（6割の確率）。蜘蛛の巣のような割れ方になる。

ヒビが伸びる・光る

- **伸びる：**入ってから `kCrackGrow`（0.08秒）かけて、起点から先へ伸びる。何区間目まで見せるかを `grow × 区間の数` で決め、最後の区間だけ途中まで描く。
- **入った直後は太く明るい：**`fresh = 1 - age × 3`。ぼんやりした光は fresh がある間だけ描く（描く回数を減らすため）。
- **先ほど薄い：**`taper` で、起点から離れるほど薄くする。
- **起点が光る：**ヒビが入った瞬間、起点に白い六角形の波紋を出す。

波紋（Ripple）

攻撃が盾に当たった所に、小さな六角形が広がって消える。当たった場所（3D）を、`right`・`up` との内積で盾の面の上の 2D 座標に直してから置く。0.1秒に1つまで（`kRippleInterval`）、同時に12個まで。

割れた時の破片（SpawnShards）

速さ	中身
横（x）	中心から外へ（角度の sin）
縦（y）	必ず上向き（120〜300 ＋ 角度の分）
奥行き（z）	カメラの方へ飛び出す

18個の三角形が、回りながら飛び、重力で少し落ち、空気の抵抗で遅くなり、0.6〜1秒で消える。三角形は辺を太い線で描いて、中を塗りつぶしたように見せている（`kShaderFilled`）。

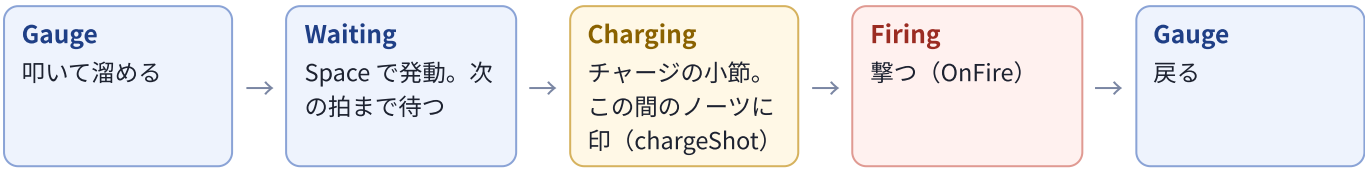
> **なぜ上へ飛ばすか：**破片が下へ落ちると、レーンの上に重なる。必ず上と奥（敵の方）へ飛ばし、落ちてくる前に消える長さにしている。

割れた後に ChartObject がすること（SetOnBreak）

- ダメージ：`40 ÷ ロングの本数 × この盾のロングの本数`
- 敵を青白く光らせる、大きな着弾（`SpawnCritical`）、画面の揺れと白い閃光
- 世界側のカメラを一瞬見上げる（`m_GazeKickTime`）、割れる音
- GameEvent の `OnShieldBreak` を放送（聞いている物がいれば反応できる）

6. 必殺技：ゲージ・弱点・ビーム

SpecialMove：状態と計算だけ



- ゲージ：1回叩くごとに `1 ÷ (0.4 × 総ノーツ数)` 溜まる。総ノーツの4割（`kFillRatio`）を叩くと満タン（判定の良し悪しは関係なし）。
- チャージ量：チャージ中のノーツの重み（WEAK 1.0／HIT 0.6／GRAZE 0.3）の平均（0〜1）。
- ダメージの式 `CalcDamage = 基本値 × (0.3 + 0.7 × チャージ量)`。全部取り逃しても3割は入る。
- チャージ中は、ロングを離してもミスにしない（`SetMissFressWindow`）。

> **なぜ分けているか：** SpecialMove は見た目もダメージも持たず、「状態が変わった」をコールバックで知らせるだけ。JudgeSystem と同じ考え方で、ルールを変えても演出を、演出を変えてもルールを触らずに済む。

WeakPoint：弱点と狙い

- 4小節ごと（`kBarsPerSwitch`）に、弱点のキー（S〜L）と、敵の上の場所をランダムに選び直す。前の場所から少なくとも 90 離す。
- 弱点にはキーの画像と、残り時間の輪を出す。残りが少ないと輪が赤くなり、半拍ごとに脈打つ。
- 必殺技の時：レーンのキーで狙いを動かし、もう一度 Space でロック。狙いが弱点と合っていると、枠が金色になる。
- 弱点のレーンの判定円に、六角形の印を出す（`SetLaneAnchor` で毎フレーム判定円の位置をもらう）。

SpecialBeam：3層のビーム

層	中身
本命の帯	太い金の帯（チャージ量で太さ 50〜140）。上にふくらむ弧。先が当たった瞬間に `OnHit` を1回
群れ	色とりどりの細い光 24本。1本ごとに、ふくらむ向き・大きさ・遅れ・速さ・太さ・色が違う。1本当たるたびに `OnSubHit`（小さい着弾）
雷	帯にまとわりつくギザギザの線 3本。0.045秒ごとに形を作り直すので、チカチカ動く

当たった時（ChartObject の `SetOnHit`）：弱点なら大ダメージ（×2.0）と大きな着弾・閃光・強い揺れ・金色に光る。違えば ×0.3 で、盾ではね返る演出（`SpawnBlocked`）。

7. 全部を支える道具

LongBeamRibbon：点の並び → 光の帯

攻撃・レーザー・盾・ヒビ・火花・必殺技のビームは、全部これ1つで描いている。

1. **点ごとに左右2つの頂点を作る：**線が進む向き `dir` と、カメラへの向きの外積で「横へ広げる向き」`side` を出す。両方に直角なので、帯はいつもカメラの方を向き、同じ太さに見える。
2. **線に沿った距離を UV に入れる：**横は 0〜1、縦は始まりからの距離。模様が流れ、つなぎ目が出ない。
3. **三角形ストリップで1回で描く：**L, R, L, R… と並べると、続く3つずつが三角形になる。
4. **シェーダーで光らせる：**`exp(-d²×6)`（細く強い芯）と `exp(-d²)`（太く柔らかい光）を足し、ノイズで揺らぎを付ける。加算合成で重ねる。

> **なぜ全部これで描くのか：**点の並べ方を変えるだけで、円・六角形・曲線・ぎざぎざ・三角形と、何でも描ける。テクスチャやモデルを用意しなくてよく、太さ・色・濃さも点ごとに決められる。

RibbonShapes：点を形に並べる

関数	すること
`CameraBasis`	ある点から見て、カメラの方を向いた面の「右」と「上」を外積で求める
`Arc`	円弧の点を足す（角度は真上が0、時計回り）。`right × sin(a) + up × cos(a)`
`Polygon`	正多角形（6なら六角形）。円弧を角の数だけの点で1周させるだけ

GameEvent：放送局

イベント	放送する所	聞いている所
`OnJudge`	ChartObject::OnJudge	HitBurstFx（破片を出す）
`OnBeat`	ChartObject::Update（拍の頭）	HitBurstFx（撃つ）
`OnAttackHit`	HitBurstFx（レーザーが届いた）	ChartObject（着弾の演出）
`OnLongEnd`	SetOnLongEnd の中	（今は誰も聞いていない）
`OnShieldBreak`	SetOnBreak の中	（今は誰も聞いていない）

- `OnJudge()` などは、関数の中の `static` 変数を参照（`&`）で返す。誰が呼んでも同じ1つの放送局になる。
- `Subscribe` で「呼んでほしい関数（ラムダ）」を名簿にしまい、`Publish` で名簿の全員を呼ぶ。
- ラムダは `[this]` で自分を覚えているので、消える前に `Unsubscribe` し、シーンの始めと終わりに `ClearAll` で名簿を空にする。

8. 読んでいて気になった所

直すかどうかは判断してほしいもの。上ほど影響が大きい。

① enemyFlash.hlsl の戻り値の型が float

`float main(in PS_IN input) : SV_Target` になっている。色は4つの値（RGBA）なので `float4 main(...)` が正しい。今の形だと、色の最初の1つ（赤）しか画面に書かれない可能性がある。敵が暗く・黒っぽく見えているなら、これが原因かもしれない。

② ロングで盾を割ると、ダメージが2回入る

押し切った時の `SetOnLongEnd`（40 ÷ ロングの本数）と、盾が割れた時の `SetOnBreak`（同じ量 × 本数）の両方が入る。割れた時だけにするか、片方を減らすか、決めておくとよい（前から残っていた課題）。

③ 盾が張られている間のタップはダメージなし

`!m_LongShield.IsUp()` の時だけダメージが入る。最初のロングの間に入れたトレースや、最後のロングの間のタップは、ダメージにならない。狙い通りならそのままでよい。レーザーは盾に当たって波紋になる。

④ HitBurstFx の上限が小さい（kMaxBursts 4、kMaxLasers 10）

上限を超えると古いものから消す。消されたレーザーが「当たった」を知らせる役（`notify`）だった場合、着弾の演出が出ない。破片のかたまりも、撃つ前に消されることがある。演出が抜けるようなら、上限を上げるか、破片の数を減らして調整する。

⑤ 必殺技がいつでも撃てる設定のまま

`SpecialMove::kDebugAlwaysReady = true`（ゲージなしでいつでも発動）。提出の前に `false` にするか、決めておく。

⑥ 小さなこと

- `Enemy::Update` は `dt = 1/60` で固定。画面の更新が速い・遅いパソコンでは、回転や点滅の速さが変わる。
- `RankColor` の GRAZE のコメントは「紫」だが、値は緑 `(0, 1, 0)`。
- AttackOrbs は今使われていない（`kTapAttackOrbs = false`）。`m_LastOrbTime` も含めて、残すか消すか。
- OnLongEnd と OnShieldBreak のイベントは、放送しているが、まだ誰も聞いていない（拍ごとのレーザーや背景の演出を足す時に使える）。